renamed, or when a method gains or loses arguments. But in this case, a method's *return type* changed, and verifying doubles can't detect that kind of incompatibility at all.

You might give up on having your unit specs detect this kind of breaking change and depend on your end-to-end acceptance specs. These will use real dependencies as much as possible and are more likely to fail on incorrect API usage.

Falling back on acceptance specs isn't the only option, however. You do have a couple of choices for making your unit specs more robust:

- Use a high-fidelity fake for the third-party API, if one is available
- Write your own wrapper around the API, and use a test double in place of your wrapper

In the next couple of sections, we're going to look at both of these approaches.

High-Fidelity Fakes

When you work with rspec-mocks, you're using fake behavior supplied by RSpec. There are other kinds of test doubles, though. You can use an alternative implementation designed to drop into your tests in place of the real thing. Because these mimic the original library's interface *and behavior* closely, we call them *high-fidelity fakes*.

For instance, the FakeFS gem makes it easy to test code that interacts with the filesystem.[8] FakeRedis acts just like the Redis data store but doesn't require a network connection or a running server.[9] If you use Braintree for credit-card processing, Fake Braintree can stand in during testing.[10]

Sometimes, a library ships with its own high-fidelity fake. The Fog gem, which wraps cloud services like Amazon's, comes with a mocks mode built in.[11] If you're developing a library that calls external services, your users will be grateful if you provide a high-fi fake. You can write one set of specs to check both the real and fake implementations using RSpec's *shared examples*. We discussed these in *Sharing Examples*, on page 124.

Fakes are particularly useful for HTTP APIs. Alas, most API clients don't include a high-fi fake. Fortunately, you can put one together fairly quickly using the VCR gem (written by Myron, one of the co-authors of this book).[12]

8. https://github.com/fakefs/fakefs
9. http://guilleiguaran.github.io/fakeredis/
10. https://github.com/highfidelity/fake_braintree
11. http://www.rubydoc.info/gems/fog/1.40.0#Mocks
12. https://relishapp.com/vcr/vcr/v/3-0-3/docs